

torchgfn: 一个 PyTorch 生成流网络库

Joseph D. Viviano^{* α} , Omar G. Younis^{* α} , Sanghyeok Choi^{* α, η} , Victor Schmidt ^{α, β} ,
Yoshua Bengio ^{α, γ, δ} , and Salem Lahlou ^{ζ, ω}

JOSEPH@VIVIANO.CA, OMAR.YOUNIS@MILA.QUEBEC, SANGHYEOK.CHOI@ED.AC.UK,
SALEM.LAHOLOU@MBZUAI.AC.AE

^{α} Mila, ^{β} Entalpic, ^{γ} Universit de Montral, ^{δ} CIFAR,, ^{ζ} MBZUAI, ^{η} University of Edinburgh, ^{ω} Work
started at Mila. ^{*}denotes co-first authorship ¹

摘要

随着生成流网络 (GFlowNets 或 GFNs) 在具有不同背景和专业领域知识的研究者中日益流行, 需要一个库来方便地测试新功能 (例如训练损失和训练策略) 与标准基准实现的对比, 或者在一个共同的环境集合上进行测试。本文介绍了 torchgfn, 这是一个旨在满足这一需求的 PyTorch 库。其核心贡献是一个模块化且解耦的架构, 将环境、神经网络模块和训练目标视为可互换的组件。这为用户提供了简单而强大的应用程序编程接口, 以促进快速原型设计和新颖的研究。提供了多个示例, 复制并统一了已发表的结果。该库可在 GitHub (<https://github.com/GFNOrg/torchgfn>) 和 pypi (<https://pypi.org/project/torchgfn/>) 上获取。

1 引言

生成流网络 (GFlowNets, GFNs; Bengio 等, 2021, 2023b) 是在具有组合结构的离散或连续样本空间上的概率模型。它们也是随机顺序采样器, 能够从目标分布生成对象, 该分布由其未归一化的概率质量函数 R 给出, 称为奖励函数。GFN 研究的迅速发展导致了多种开源实现。虽然这表明了一个健康的生态系统, 但现有的大多数开源代码都是针对特定项目的定制版本, 而现有的库通常针对特定领域 (例如, 分子生成²), 完整的应用围绕复杂的实验框架设计³, 或者更侧重于性能而非可扩展性或易用性⁴。我们介绍了 torchgfn, 这是首个提供通用、高度模块化且用户导向的基于 PyTorch (Paszke 等, 2019) 工具包的库, 该工具包从一开始就旨在便于扩展和修改。其设计理念在于严格分离关注点, 这不仅使研究人员能够复制现有工作, 更重要的是, 加速了 GFN 研究在广泛应用领域的下一代发展。它将环境定义、采样过程以及用于 GFN 损失的参数化进行了解耦。该库旨在向新用户介绍 GFNs 及其连续变体 (Lahlou 等, 2023; Sendera 等, 2024; Zhang 等, 2024), 并促进新算法的发展。该库附带了许多 Gym 中的示例环境, 这些环境捕捉到了各种 GFN 应用场景。一些示例包括: 1) 所有状态均为终止状态的离散环境 (例如, HyperGrid (Bengio 等, 2021)); 2) 所有轨迹长度相同但只有某些状态为终止状态的离散环境 (例如, DiscreteEBM); 3) 简单自回归设置 (例如, BitSequence); 4) 具有状态依赖动作空间的连续环境 (例如,

- ordered by geographic proximity to Saint Joseph's Oratory of Mount Royal at the time of publication.
Co-first authors agree that they can list their names in any order on their CVs.
- github.com/recursionpharma/gflownet
- github.com/alexhernandezgarcia/gflownet, used for *e.g.*, AI4Science et al. (2023).
- github.com/d-tiarkin/gfnx, a JAX-based library supporting discrete environments.

Box) ; 5) 图的离散采样 (例如, BayesianStructure (Deleu 等人, 2022) 和 Ring) ;
 6) 基于扩散的采样器用于连续分布 (例如, DiffusionSampling (Sendera 等, 2024 年)) 。此列表无法详尽无遗, 因为随着 torchgfn 被应用于各种应用领域, Gym 将由维护者和社区积极开发。这些示例有助于用户学习 GFN 的理论, 复现已发布的环境, 展示库的正确使用方法, 并提供如何扩展基类以适应特定应用场景的例子。

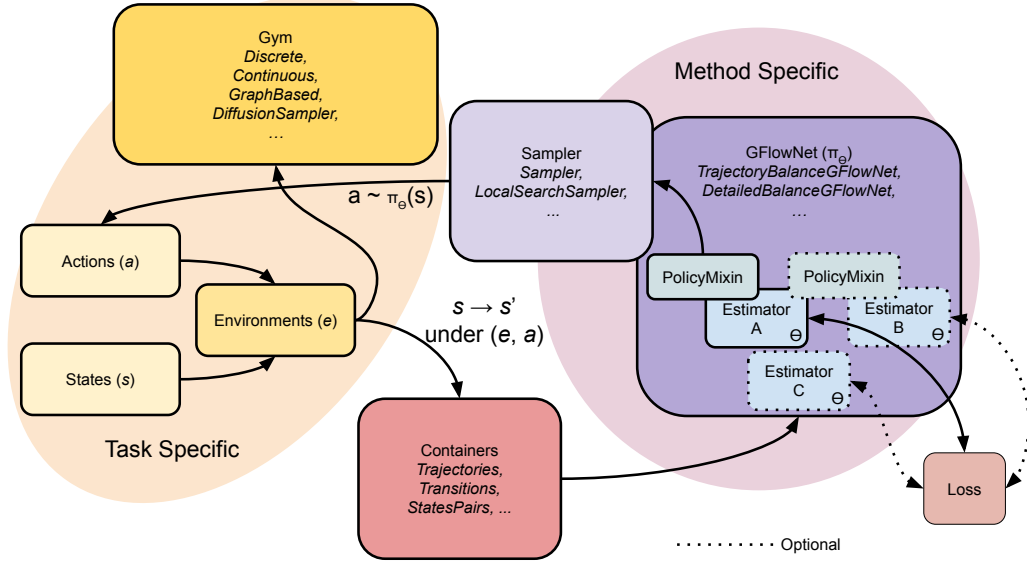


图 1: 模块化 torchgfn 存储库结构的示意图。状态和动作是最顶层的抽象, 用于在无状态环境之间进行接口。容器是通用对象, 用于保存从采样器中抽取的状态-动作序列。这些通过策略混合器直接与 GFlowNet 子类 (例如 TrajectoryBalanceGFlowNet) 的具体训练目标参数化接口。参数化由一个或多个估计器 (pytorch nn.Module 包装器) 处理, 这些估计器表示学习到的功能近似器 (例如, 策略 π , 状态流), 使用 GFlowNet 的损失进行训练。一组环境通过 Gym 模块提供。

2 库架构

库层次的概念图如图 1 所示 (请参见附录图 2 以详细了解库的完整依赖结构)。总体而言, 在有向无环图 (DAG) 上定义的马尔可夫决策过程 (MDP) 通过状态和动作之间的交互 (通过无状态环境) 实现, 生成新的状态实例。由于这些元素与训练任务紧密相关, 它们在环境中的一处由用户定义。状态-动作容器与由估计器参数化的 GFlowNets 交互, 通过采样器抽样动作以探索 DAG。估计器使用 GFlowNet 的 `loss()` 方法进行训练, 而这些估计器与采样器之间的接口由策略混合器调解, 使通用采样器能够与任何类型的估计器架构一起工作。关于每个库元素的详细说明可在附录 B 中找到, 最新文档可在在线文档 ⁵ 中获取。仓库教程中也有很多工作示例 ⁶。

5. <https://torchgfn.readthedocs.io/en/latest/>

6. <https://github.com/GFNOrg/torchgfn/tree/master/tutorials>

环境 e 是无状态对象，具有产生状态转换的 step 函数 s'

从 s 给定一个动作 a , i.e., $s \rightarrow s'$ 在 (e, a) 下。它们的状态无关构造便于轻松扩展。状态 s 是 GFlowNet 训练对象的基本构建块，包含状态级别的元数据，如禁止不希望或不可能的状态转换的掩码。它们可以是任何数据类型，但必须能够转换为与 pytorch 和/或 torch geometric 兼容的格式。动作 a 表示代理在构建组合对象时的内部动作。容器在采样过程中将状态和动作包装成给定 GFlowNet 所需的正确格式。预处理器负责将状态中包含的数据类型转换为与您定义的估计器兼容的类型。估计器是 nn.Modules 或其他 pytorch 模块抽象的封装，将预处理后的状态转换为动作分布。采样器定义了如何在每个状态下采样动作，依赖于估计器生成要从中采样的分布。**GFlowNets** 封装了所需的预处理器、估计器、采样器以及与损失相关的逻辑，以生成训练好的采样器。除了基于 nn.Module 的支持外，我们还提供了使用 torch geometric 进行图对象采样的支持，请参见第 C 节以获取更多详细信息。这展示了如何构建和训练基于轨迹平衡的 GFN (Malkin 等, 22 年)：

```
1 env = HyperGrid(ndim=4, height=8, reward_fn_kwargs={"R0": 0.01})
2 prep = KHotPreprocessor(ndim=env.ndim, height=env.height)
3 module_PF = MLP(input_dim=prep.output_dim, output_dim=env.n_actions)
4 module_PB = MLP(input_dim=prep.output_dim, output_dim=env.n_actions - 1)
5 gfn = TBGFlowNet( # Define the GFlowNet.
6     pf=DiscretePolicyEstimator(
7         module_PF, env.n_actions, is_backward=False, preprocessor=prep),
8     pb=DiscretePolicyEstimator(
9         module_PB, env.n_actions, is_backward=True, preprocessor=prep),
10 )
11 sampler = Sampler(estimator=gfn.pf)
12 optimizer = torch.optim.Adam(gfn.parameters(), lr=1e-3)
13
14 for i in (pbar := tqdm(range(1000))): # Training.
15     trajectories = sampler.sample_trajectories(env=env, n=16)
16     optimizer.zero_grad()
17     loss = gfn.loss(env, trajectories)
18     loss.backward(); optimizer.step()
```

To instead train a SubTBGFlowNet, simply change the parameterization of the GFlowNet:

```
1 module_logF = MLP(input_dim=prep.output_dim, output_dim=1)
2 gfn = SubTBGFlowNet( # Define the SubTrajectory GFlowNet.
3     pf=DiscretePolicyEstimator(
4         module_PF, env.n_actions, is_backward=False, preprocessor=prep),
5     pb=DiscretePolicyEstimator(
6         module_PB, env.n_actions, is_backward=True, preprocessor=prep),
7     logF=ScalarEstimator(module=module_logF, preprocessor=prep),
8 )
```

2.1 设计理念与其他库的对比

torchgfn 被设计为领域无关的通用库，提供开箱即用的支持，适用于离散、连续和基于图的环境，并通过统一的应用程序编程接口优化了可扩展性。这种灵活性使其成为研究 GFlowNet 框架本身的理想平台——创新可以在执行速度和优化不是首要考虑的已知环境中进行测试。recursionpharma/gflownet (Bengio 等, 2023a, 以下简称 recursion) 明确专门用于图和分子生成，利用 torch geometric 和 networkx 作为其核心数据表示，而 alexhernandezgarcia/gflownet (Hernandez-

加西亚等人（2023 年，以下简称 gflownet）在复杂科学应用（如晶体生成）以及标准环境方面进行了深入设计，而基于 JAX 的 gfnx（提亚普金等人，2025 年）则提供了离散环境和较少的抽象，以换取具有显著速度优势的即时编译采样循环。尽管各自在其领域内功能强大，但每个库的专业化也为其适应某些问题类型和基本 GFlowNets 方法设置了障碍（参见表 1）。torchgfn 的核心抽象（第 B.1 至 B.7 节）是可组合的对象，用户可以对其进行修改以干净地实现特定或新颖的方法。例如，GFN 训练目标本身就是一个可互换的 GFlowNet 对象，明确区分了算法与神经网络模型（估计器）。采样器和估计器逻辑也通过可交换的 PolicyMixin 接口得以解耦。

表 1: GFlowNet 库的高层次比较。

Library	torchgfn Pytorch	recursion Pytorch	gflownet Pytorch	gfnx JAX
Philosophy	Modular & Decoupled	Graph (Molecule)-Focused	Environment-Centric	Low-Level & Fast
Envs	Stateless	Stateless	Stateful	Stateless
Focus	General-Purpose	Graph & Molecule	General-Purpose	Discrete Envs
Batching	torch.Tensor & torch_geometric.Data	torch_geometric.Batch	Custom Batch Class	jax.vmap
Loss Impl.	Classes	Classes	Methods in Agent	User-Defined

与替代方案相比，torchgfn 处于性能和易用性的中间地带，旨在满足希望快速启动想法且不太关心最快训练时间的用户需求（表 2，完整的基准测试结果请参见附录 E.3）。

3 未来工作与结论

我们在未来开发中打算解决的一些局限性包括 1) 简化 States、Actions 和 Envs 的可扩展模块化设计，使环境定义更加容易，并与 torch.compile 更加兼容，2) 扩展库以包含与语言模型和 AI 科学社区相关的更复杂状态空间的真实任务，3) 原生支持分层采样，通过在 Trajectories 中存储采样元数据实现 7 4) 支持多目标元环境，5) 包含强化学习基线。总之，torchgfn 采用了模块化架构，使研究人员能够将新 GFN 算法作为独立且可导入的类进行开发和测试，我们认为这满足了 GFN 社区未被满足的需求。它代表了对 GFN 构建块的重要改进，将其转化为一组文档齐全且经过测试的抽象，简化了实验过程，提高了可读性和可重复性。我们的项目已经有许多外部贡献者，并且正在建立一个具有适当贡献标准的动态开发者社区来支持 GFNs 8 我们希望它成为基础研究的社区标准。

表 2: 在 Hypergrid（批大小 128）上，跨 3 个随机种子的每迭代时间平均值。

Scenario	Library	Time (ms)	Mem (MB)	Params
Small	gflownet	141.4	24	70,915
Small	torchgfn	30.8	24	71,430
Small	gfnx	2.5	182	71,429
Medium	gflownet	223.0	26	83,717
Medium	torchgfn	77.8	43	84,746
Medium	gfnx	5.9	220	84,745
Large	gflownet	364.7	37	100,101
Large	torchgfn	113.3	55	101,130
Large	gfnx	10.8	361	101,129

7. 目前，用户需要为这些用例子类化 PolicyMixin 和 Trajectories 类。8. github.com/GFNOrg/torchgfn/blob/master/.github/CONTRIBUTING.md

致谢及资金披露

作者谨向提交有价值的拉取请求的社区成员表示衷心感谢：Alexandre Larouche、Mike Arpaia、Idriss Malek、Abhijith Sharma、Ali Parviz、Etienne Collin、Emir Ceyani、Joseph Bunao、Aya Laajil、Sanchit Misra、Chirayu Haryan 及 Inel Djafar。我们还要感谢 Mo Tiwari、Edward Hu、Tristan Deleu、Daniel Jenson、Eric Elmoznino、Nikolay Malkin、Emmanuel Bengio、Alex Hernandez-Garcia 及 Anja Surina 提供的宝贵讨论和反馈。这项工作得到了 Intel、Samsung 以及魁北克经济与创新部（MEI）的支持。

参考文献

- Mila AI4Science, Alex Hernandez-Garcia, Alexandre Duval, Alexandra Volokhova, Yoshua Bengio, Divya Sharma, Pierre Luc Carrier, Yasmine Benabed, Michał Koziarski, and Victor Schmidt. Crystal-gfn: sampling crystals with desirable properties and constraints. *arXiv preprint arXiv:2310.04925*, 2023.
- Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Emmanuel Bengio et al. recursionpharma/gflownet. <https://github.com/recursionpharma/gflownet>, 2023a.
- Yoshua Bengio, Salem Lahlou, Tristan Deleu, Edward J Hu, Mo Tiwari, and Emmanuel Bengio. Gflownet foundations. *Journal of Machine Learning Research (JMLR)*, 24(210):1–55, 2023b.
- Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. TorchRL: A data-driven decision-making library for PyTorch, 2023.
- Tristan Deleu, António Góis, Chris Emezue, Mansi Rankawat, Simon Lacoste-Julien, Stefan Bauer, and Yoshua Bengio. Bayesian structure learning with generative flow networks. *Uncertainty in Artificial Intelligence (UAI)*, 2022.
- Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *nature*, 585(7825):357–362, 2020.
- Alex Hernandez-Garcia et al. alexhernandezgarcia/gflownet. <https://github.com/alexhernandezgarcia/gflownet>, 2023.
- Minsu Kim, Taeyoung Yun, Emmanuel Bengio, Dinghuai Zhang, Yoshua Bengio, Sungsoo Ahn, and Jinkyoo Park. Local search gflownets. *International Conference on Learning Representations (ICLR)*, 2024.
- Aya Laajil, Abduragim Shtanchaev, Sajan Muhammad, Eric Moulines, and Salem Lahlou. Curriculum-augmented GFlowNets for mRNA sequence generation. *arXiv preprint arXiv:2510.03811*, 2025.
- Salem Lahlou, T. Deleu, Pablo Lemos, Dinghuai Zhang, Alexandra Volokhova, Alex Hernandez-Garcia, L’ena N’ehale Ezzine, Y. Bengio, and Nikolay Malkin. A theory of continuous generative flow networks. *International Conference on Machine Learning (ICML)*, 2023.

- Kanika Madan, Jarrod Rector-Brooks, Maksym Korablyov, Emmanuel Bengio, Moksh Jain, Andrei Nica, Tom Bosc, Yoshua Bengio, and Nikolay Malkin. Learning GFlowNets from partial episodes for improved convergence and stability. *International Conference on Machine Learning (ICML)*, 2023.
- Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. *Neural Information Processing Systems (NeurIPS)*, 2022.
- Ling Pan, Nikolay Malkin, Dinghui Zhang, and Yoshua Bengio. Better training of GFlowNets with local credit and incomplete trajectories. *International Conference on Machine Learning (ICML)*, 2023.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Marcin Sendera, Minsu Kim, Sarthak Mittal, Pablo Lemos, Luca Scimeca, Jarrod Rector-Brooks, Alexandre Adam, Y. Bengio, and Nikolay Malkin. Improved off-policy training of diffusion samplers. *Neural Information Processing Systems (NeurIPS)*, 2024.
- Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Daniil Tiapkin, Artem Agarkov, Nikita Morozov, Ian Maksimov, Askar Tsyganov, Timofei Gritsaev, and Sergey Samsonov. gfnx: Fast and scalable library for Generative Flow Networks in JAX. *arXiv preprint arXiv:2511.16592*, 2025.
- David W. Zhang, Corrado Rainone, M. Peschl, and R. Bondesan. Robust scheduling with GFlowNets. *International Conference on Learning Representations (ICLR)*, 2023.
- Dinghui Zhang, Ricky TQ Chen, Cheng-Hao Liu, Aaron Courville, and Yoshua Bengio. Diffusion generative flow samplers: Improving learning signals through partial trajectory optimization. *International Conference on Machine Learning (ICML)*, 2024.

附录 A. 库结构

完整的内部库依赖关系分解可以在图 2（截至 v2.3.0 版本）中看到。高级库结构如下所示：

- `actions.py`: 表示动作的对象。
- `states.py`: 表示状态的对象。
- `env.py`: 所有环境的基础类。
- `containers/`: 存储状态、动作、估计器输出及其他辅助元数据的对象集合，用于训练 GFlowNet。这包括回放缓冲区。
- `preprocessors.py`: 将状态转换为估计器可以处理的对象的函数。
- `estimators.py`: 以估计器和策略混合类形式封装的 `nn.Module` 包装器，包含 GFlowNet 的学习参数及其与采样器和对数概率计算的接口。
- `samplers.py`: 包含采样逻辑的类。
- `gflownet/`: 封装可训练 GFlowNet 完整参数化的类，包括损失函数和采样过程。
- `gym/`: 环境集合。
- `utils/`: 各种实用工具函数。

附录 B. 代码库组件

B.1 环境

在 `torchgfn` 中，环境 e 是无状态对象，具有一个给定动作 a 和状态 s 产生转换后状态 s' , *i.e.*, $s \rightarrow s'$ 的 `step` 函数，该过程遵循 (e, a) 。环境的无状态特性是一种有意的设计选择，它允许在大规模多节点设置中进行训练时实现并行计算，在这种设置中，可以将对环境的查询（用于下一状态计算和/或奖励计算）广播到所需的任何专用计算节点，从而解耦可能由 GFlowNet 动作采样显著快于环境查询所引起的计算瓶颈。定义环境需要用户指定 MDP 的关键组件，包括初始状态 s_0 和终止状态 s_f 的表示，以确保 MDP 结构为有向无环图 (DAG)，一个包含管理动作掩码逻辑的 `States` 类工厂（即，跟踪并强制执行当前状态下所有允许的动作），最后是一个定义可被环境识别的采样动作表示的 `Actions` 类工厂。提供了特定的环境抽象，例如，`DiscreteEnv` 允许通过将总动作数作为属性来轻松指定动作空间。环境必须实现 `log_reward()` 或 `reward()` 方法，这些方法将在每个终止状态（即，DAG 中只有 s_f 作为子节点的状态）以及如果环境允许且 GFlowNet 将使用中间奖励进行训练的情况下，在非终止状态调用 (Pan et al., 2023)⁹。在定义环境时，除了 s_0 ，用户还可以选择定义一个表示汇状态 s_f 的张量，仅用于填充不完整的轨迹。如果没有指定，`sf` 将设置为与 s_0 形状相同的张量，其中填充 $-\infty$ 。

9. 环境设计者应留意由于奖励值尺度而产生的数值稳定性问题。

对于离散环境 (DiscreteEnvs)，用户可以定义一个获取状态索引的方法 `get states indices()`，该方法为每个状态分配一个唯一的整数编号，并定义一个 `n states` 属性，返回环境中状态的数量（不包括 s_f ）。还可以实现 `get terminating states indices()` 函数，用于唯一标识环境中的终止状态，这对表格估计器是有帮助的。其他属性和函数也可以实现，例如日志分区或真实分布属性 `true dist`。

B.2 状态

状态 是 GFlowNet 训练对象的基本构建块，如转换和轨迹，在这些对象上执行损失操作。提供的抽象 **States** 类必须针对每个环境进行子类化以定义 s_0, s_f ，并定义单个批次元素的状态形状。这使得 **States** 实例能够包含特定于环境的状态信息，从而使环境本身保持无状态。**States** 对象是一组状态的集合，通常存储为基于 `torch.tensor` 的批次以实现高效处理。在环境需要异构数据的情况下，例如具有不同数量节点和边以及全局属性的图，`torchgfn` 利用 `numpy.Array` (Harris 等, 2020) 和 `tensordict` (Bou 等, 2023)，这是 PyTorch 生态系统中的一个库。Numpy 数组用于高效管理 torch 几何对象，而 `TensorDict` 是一个类似字典的容器，可以容纳不同形状和类型的张量，并允许统一的批处理和设备管理操作，从而产生更干净、更高效的代码，有效地管理多个外部 pytorch 库，以实现训练 GFlowNet 的目标。

掩码：并非所有动作在所有状态下都是可行的——这一约束通过状态的掩码属性来处理。`DiscreteStates` 和 `GraphStates` 对象（分别针对状态表示为离散变量张量和图的环境的状态实现）具有前向掩码和后向掩码，分别指定了每个状态下允许的动作以及能够生成该状态的动作。掩码可以作为 `States` 类的一个属性实现，并通过环境中定义的更新掩码方法保持更新，也可以作为属性实现，以便在每个状态下按需生成。虽然从技术上讲可以在没有掩码的情况下训练 GFlowNet，但在训练过程中采样的无效轨迹数量可能如此之多，以至于损失函数无法在合理的时间内收敛，因此几乎总是实用的必要条件和状态实现的重要组成部分。

数据类型：值得注意的是，`States` 对象中包含的数据可以采取任何形式（例如字符串、numpy 数组），但 `torchgfn` 只明确支持某些形式，这些形式允许利用 pytorch 进行高效的 GFN 训练，包括 `torch.Tensors` 和 `torch geometric` 的图数据对象。为了处理其他表示形式，用户需要在定义 GFlowNet 时向 `Estimator` 提供一个自定义的预处理器，将这些数据结构转换成批量张量。

B.3 动作

动作 代表代理构建组合对象的内部动作：它们对应于状态之间的转换 $s \rightarrow s'$ 。提供了一个抽象的 **Actions** 类。虽然它会自动为简单的离散环境进行子类化，但在其他情况下必须手动进行子类化，因为动作需要了解状态表示，从而了解环境。在 `DiscreteEnv` 情况下，动作只是一个表示 0 到 $n_{actions} - 1$ 之间索引的整数。此外，允许轨迹提前终止的环境需要一个与终止动作对应的退出动作张量（对于离散环境为 $[n_{actions} - 1]$ ），除非所有轨迹都有固定的长度。对于离散环境，动作集 $\{0, \dots, n_{actions} - 1\}$ 还包含一个第 $(n_{actions})$ 个退出或终止动作（*i.e.*, $s \rightarrow s_f$ ），对应于索引 $n_{actions} - 1$ 。

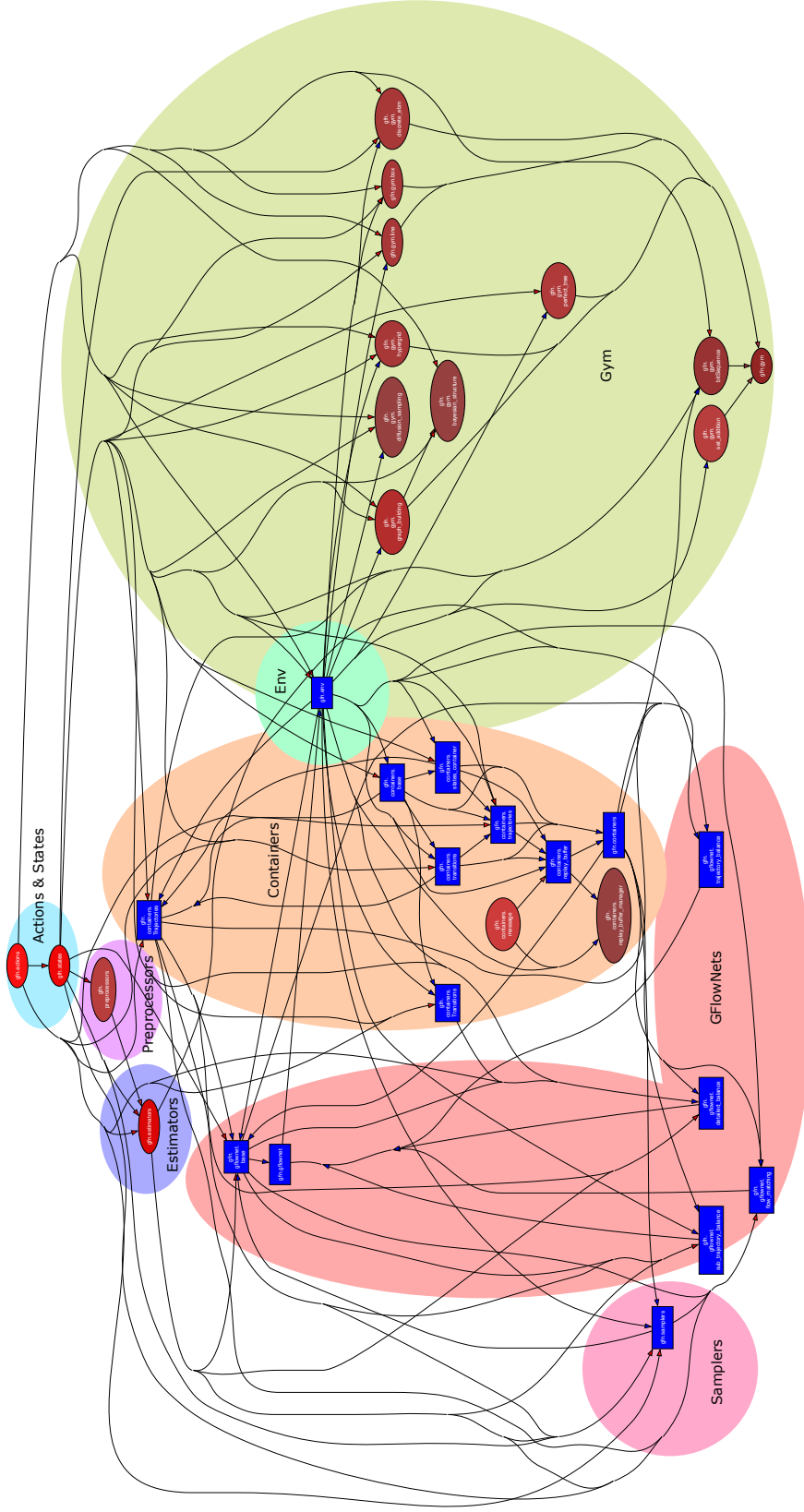


Figure 2: torchgfn dependency graph as of v2.3.0. States and Actions are top-level abstractions used to interface between the stateless `Env` and a `GFlowNet`. Containers wrap `States-Actions` sequences as either `Transition` or `Trajectory-level` information to be used by the remainder of the library. They're drawn from `Samplers` via `Estimators` (wrappers for `pytorch.nn.Module`) representing function approximators (*e.g.*, policies, state flows) housed within the `GFlowNet` subclass (*e.g.*, `TrajectoryBalanceGFlowNet`). The interface between `Samplers` and `Estimators` is mediated by a `PolicyMixin`. The `GFlowNet` subclass contains all loss-relevant logic. `Environments` are made available through the `Gym` module.

B.4 容器

容器 在采样过程中封装状态和动作——它们包含了训练 GFlowNet 所需的所有元素，形式为抽象如转换或轨迹。例如，这些可以保存在轨迹采样期间估算器的输出，以便于使用离策略方法训练模型时计算对数概率，以及其他可能在特定参数化下计算损失时有用的元数据。容器是状态的集合，以及其他信息，如奖励值或密度 $p(s' | s)$ 。提供了三种容器：• 转换，表示一批转换 $s \rightarrow s'$ 。

- 轨迹，表示一批完整的轨迹 $s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n \rightarrow s_f$ 。
- 状态容器，表示一批状态（例如，父状态和子状态，即 (s_p, s_c) ，用于流匹配损失 Bengio 等 (2021)。

这些容器可以通过 `States` 对象实例化，也可以初始化为空容器，然后在运行时填充，允许使用 `ReplayBuffer` 类，以实现经验回放，从而在使用离策略方法训练 GFlowNet 时通过不相关的轨迹稳定训练。它们继承自基础容器类，指示了一些有用的方法。大多数情况下，需要采样完整的轨迹。从一批轨迹中，可以使用 `Trajectories.to_states()` 和 `Trajectories.to_transitions()` 定义一批状态和一批转换，以便于使用边分解或状态分解损失训练 GFlowNets。这些排除了添加到轨迹批次中以实现高效批处理的无意义转换和虚拟状态。

B.5 估算器

Training GFlowNets using a `pytorch` optimizer requires at least one **Estimator** (an abstract subclass of `torch.nn.Module`). In addition to the usual `forward` method, **Estimators** implement other useful attributes that interface with the rest of the `torchgfn` library, for example, ensuring `torch.nn.Module` outputs have the correct output dimensions for the action space, or the required logic to convert module outputs into probability distributions.

For all **Estimators**, the `forward` function accepts a **States** object. Neural network estimators require tensors in a particular format, and therefore one may need to define a **Preprocessor** object as part of the environment that transforms the **States** representation into something compatible with the **Estimator** in question (Section B.1).

As an example, a **DiscretePolicyEstimator** is an **Estimator** that can be used to define the policies $P_F(\cdot | s)$ and $P_B(\cdot | s)$ for discrete environments. At initialization, when `is_backward=False`, the required output dimension is `n_actions`, and when `is_backward=True`, it is `n_actions - 1`¹⁰. These numbers represent the logits of a categorical distribution. The corresponding `to_probability_distribution()` function transforms the logits by masking illegal actions (according to the forward or backward masks contained in the **States** instance), and returns a categorical distribution. Masking is accomplished by setting the corresponding logit to $-\infty$. The function also includes exploration parameters, in order to define a tempered version of P_F , or a mixture of P_F with a uniform distribution. Other simple examples for *discrete environments* include the **Tabular** module, which implements a lookup table that can be used instead of a neural network, and a **UniformPB** module, which implements a uniform backward policy.

For *non-discrete environments*, the user needs to specify their own policies P_F and P_B . Each module should accept a batch of **States** and return batched parameters of `torch.Distributions`. The distribution required depends on the environment and may also depend on the previous state

10. 向后策略没有退出动作。

本身。我们的教程和示例包含了各种实现细节——例如，在 Box 环境中，前向策略的支持要么在一个四分之一圆盘上，要么在一个弧圈上，使得角度和半径（对于四分之一圆盘部分）是从 Beta 分布混合中缩放的样本¹¹。to_probability_distribution() 函数负责将参数输出转换为实际的批量分布对象，该对象至少实现了 sample() 和 log_prob() 函数。

递归策略：对于维护隐藏状态的序列模型（例如 RNN 或自回归变换器），torchgfn 提供了 RecurrentDiscretePolicyEstimator，该估计器扩展了标准策略接口，增加了显式的携带管理功能。估计器的前向传递接受状态和携带字典作为输入，返回更新后的对数概率和下一个携带状态。通过 PolicyMixin 架构，这些递归估计器可以无缝地与库的采样和概率计算工具集成，通过统一的展开 API 自动处理每一步工件跟踪和上下文管理。

B.6 采样器

采样器对象定义了在每个状态下如何进行动作采样。它们需要一个实现了 to_probability_distribution() 方法的估计器。它们还包括一个 sample_trajectory() 方法，可以从给定的一组初始状态开始采样一批轨迹或 s_0 。对于离线策略采样，可以直接在初始化采样器时传递 to_probability_distribution() 的参数。采样器可以遵循任何逻辑，用户可以定义自己的采样器，并将其传递给 GFlowNet 以启用新功能。例如，请参阅包含的 LocalSearchSampler，它通过迭代破坏并重新从高奖励轨迹中采样来促进高效的局部探索（Kim 等，2022 年）。为了适应多样化的策略架构，torchgfn 使用了一个 PolicyMixin 框架，为矢量化（无状态）和非矢量化（递归）估计器提供了一致的展开 API。此混合提供了初始化展开上下文、计算分布和评估对数概率的方法，使采样器和概率计算器能够忽略估计器是否维护内部状态。每一步的工件，如对数概率和估计器输出，会自动在 RolloutContext 中跟踪，该上下文由混合在整个轨迹生成过程中管理。

B.7 GFlowNet 和损失函数

GFlowNets 可以使用不同的损失函数进行训练，每种损失函数需要不同的参数化设置，因此这些作为统一的 GFlowNet 对象在库中可用。torchgfn 的一个关键架构特征是将 GFlowNet 损失函数视为可互换的对象。研究者可以定义一组用于策略和状态流的估计器，并通过简单地将这些模块传递给不同的 GFlowNet 类实例（例如 TrajectoryBalanceGFlowNet、DetailedBalanceGFlowNet 等），无缝切换训练目标。这种组合方法促进了更快和更干净的实验。它是一个元估计器，包含一个或多个估计器，其中至少有一个实现了 to_probability_distribution() 函数。它们还必须实现一个 loss() 函数，该函数根据损失类型接受状态、转换或轨迹作为输入。实现的损失包括流匹配损失（Bengio 等，2021 年），详细平衡损失（Bengio 等，2023b 年）及其修改变体（Deleu 等，2022 年），轨迹平衡损失（Malkin 等，2022 年），子轨迹平衡损失（Madan 等，2023 年），以及对数分区方差损失（Zhang 等，2023 年）。

11. 提供的 Box 示例展示了一个复杂的场景，通常情况下，用户自定义环境不需要如此详细的细节。

附录 C. 抽样图对象

在 `torchgfn` 中，可以通过使用 `torch.Tensor` 表示（例如构建邻接矩阵）来抽样图，这在一般情况下可能是次优且相当有限的，或者也可以使用包含的 `GraphEnv` 和 `GraphStates`，它们支持在典型 `torchgfn` 训练设置中的核心操作。这允许使用基于 GNN 的估计器构建估计器，例如 `GraphEdgeActionGNN`，该估计器假设固定数量的节点并仅采样它们之间的边。构建图的动作空间可以是任务所需的任何有效图动作的子集，从而为需要基于图的状态的环境设计者提供最大的灵活性。我们在库中有多个示例，例如 `train graph ring.py` 和 `train bayesian structure.py`，展示了它们的使用，并将继续扩展此功能。

附录 D. v2 新增内容

v2 版本标志着库的整体成熟度有了显著提高，并引入了一些主要改进，相较于 2022 年的初始版本：

- **图生成支持**：通过将 `torch geometric` 集成到我们的 `GraphStates` 类中，实现了一流的图生成支持，利用基于图的状态。
- **条件 GFlowNets**：通过传递给估计器的条件张量实现了条件生成，如 Laajil 等人（2025 年）所使用的。
- **自定义采样器**：支持用户自定义的采样策略，例如 `LocalSearchSampler` 用于高效的局部探索（Kim 等，2024 年）。
- **循环策略**：一种 `PolicyMixin` 架构，统一了无状态和循环估计器，通过共同的展开 API，使序列模型（RNN、LSTM、GRU、变换器）能够在轨迹生成过程中维持隐藏状态。
- **扩散采样**：通过 `DiffusionSampling` 环境支持基于扩散的采样器，实现 GFlowNet 对连续分布的训练（Sendera 等，2024）。
- **丰富的生态系统**：扩展并验证的教程与成熟的在线文档一同在 <https://torchgfn.readthedocs.io> 提供。

附录 E. 开发者体验与项目成熟度

对于一个开源库而言，开发者的体验质量对于其被采纳和长期成功至关重要。`torchgfn` 提供了一个丰富的学习生态系统，包括在 `ReadTheDocs` 上的文档、快速入门指南、配有可运行脚本和交互式 Jupyter 笔记本的详细教程。这种全面的方法显著降低了新用户的入门门槛。该库采用成熟的测试套件 `pytest`，并通过 `pre-commit` 钩子强制执行代码质量。由 GitHub Actions 管理的持续集成（CI）流水线会自动运行测试、验证教程并处理向 PyPI 发布。这一级别的自动化和质量保证对于一个生产就绪的库来说非常重要。`torchgfn` 利用 `poetry` 进行依赖管理，通过 `pyproject.toml` 文件实现。这种方法确保了可重复的环境并简化了包的分发，符合 Python 生态系统中的当前最佳实践。

E.1 大幅提高模块化以支持扩展的算法工具包

近期的研发工作重点在于大幅提高库的模块化程度，以支持新功能的引入。`PolicyMixin` 框架体现了这种模块化设计：它

提供了一个统一的接口，用于处理向量化（无状态）和非向量化（递归）策略，通过一个小型的 rollout 应用程序编程接口来计算分布、对数概率，并跟踪每一步的工件。这使得相同的采样器和概率工具可以驱动不同的估计器家族——离散型、基于图的、条件型和递归型——而无需专门的集成代码。同样，引入条件估计器、LocalSearchSampler (Kim 等, 2024 年) 以及扩散采样的支持只需要对库进行局部添加或修改。我们认为这种模块化将在未来成为一项重要资产，随着社区开发新的方法或应用 GFNs。

附录 E.2 丰富的教程和可复现性中心

torchgfn 库还包含大量的交互式教程，这些教程通过持续集成进行测试，确保它们在库演进过程中继续正常工作¹²，以及一些示例，这些示例作为学习 GFlowNets 基础知识的丰富资源，以及最近发表结果的再现，例如 (Kim 等, 2024 年; Deleu 等, 2022 年; Lahlou 等, 2023 年; Malkin 等, 2022 年; Zhang 等, 2023 年)¹³。这些示例也在持续集成下进行测试。我们设想这个库将支持一个集中化的、标准化的 GFlowNet 相关再现和基于代码的学习资源集合，随着任务套件的增长，涵盖文献中的各种应用，这些资源将对社区普遍有用。

附录 E.3 与替代库的性能比较

我们比较了 torchgfn、gflownet 和 gfnx 在截至 2026 年 2 月 10 日的四个环境家族中的九种场景配置。我们在三个尺度上运行 HyperGrid: 小 (dimension=2, height=8, 状态空间= $8^2 = 64$), 中 (dimensions=4, height=16, 状态空间= $16^4 = 65,536$), 和大 (dimension=4, height=32, 状态 space= $32^4 = 1,048,576$); Ising 在两个尺度上: 6x6 和 10x10; Box2D 使用学习到的或均匀的反向策略; 以及小规模 BitSequence (序列 size=8, 词 size=1, modes=2) 和中等 (序列 size=8, 词 size=2, modes=4)。所有场景均使用轨迹平衡损失。在所有运行中, GFN 策略架构尽可能接近匹配, 对库特定决策有一定的容忍度 (例如, 退出动作的处理), 导致参数计数变化的标准差为 $\pm 10.2\%$ 。每次运行开始时有 50 次预热迭代, 这些迭代不计入时间测量。这允许 PyTorch CUDA 核函数进行缓存, 并使 JAX 函数进行即时编译, 确保后续测量反映稳态性能。100 次计时迭代中的每一次都独立使用 `time.perf_counter()` 进行测量, 在每次迭代前后进行显式设备同步 (PyTorch 使用 `torch.cuda.synchronize()`; JAX 使用 `jax.block_until_ready()`), 以消除异步执行工件。在可能的情况下记录每阶段的时间 (采样、损失计算、反向传播、优化器步骤); gfnx 仅报告总步时间, 因为其即时编译的训练步骤无法无开销地进行工具化。每个场景在批大小 32、128 和 512 下运行, 以表征扩展行为。对于 Ising 10 $\times$ 10, 在批大小 512 时跳过 gflownet, 因为其动作索引存在禁止的 $\mathcal{O}(\text{批} \times \text{动作空间} \times \text{轨迹长度})$ 扩展。所有实验在 3 个随机种子 (0, 1, 2) 上重复进行。我们报告跨种子的平均迭代时间, 标准差捕捉种子间的变异性。对于每个 (场景, 批大小, 库, 种子) 组合, 我们记录: 每迭代墙钟时间的平均值和标准差、每阶段时间分解、峰值设备内存分配和总可训练参数数量, 这些数据展示在表 3 中。在训练前后检查了对数分区函数估计值, 作为优化正确执行的合理性检查。在图 3 中出现了一种普遍趋势: torchgfn 处于 gflownet 和 gfnx 之间的舒适中间地带, 运行时间大约为 $2 - 10\times$ 比前者更快且达到 $10 - 20\times$ 更慢

¹² github.com/GFNOrg/torchgfn/tree/master/tutorials/notebooks

¹³ github.com/GFNOrg/torchgfn/tree/master/tutorials/examples

相较于后者而言，运行时间的比较主要受到批大小、轨迹长度以及环境的动作空间的影响，在我们的伊辛模型实验中这一点尤为明显。gflownet 库具有较低的固定开销，因此在小批量情况下表现出色，但该库的一些设计决策使得用户无法利用沿批次维度的张量向量化，并导致运行时间随批大小线性增长。对于较大的动作空间，还有一些操作在 $O(batch_size \times action_space)$ 时间内执行，这会降低性能。另一方面，gfnx 能够利用即时编译 (JIT)，在所有批大小下相对于 torchgfn 提供了显著的速度提升。这一过程使我们认识到 torchgfn 中存在多个与使用 torch.compile 不兼容的设计决策，比如我们 States 和 Actions 类的结构。我们计划通过一系列改进来解决这些问题，以实现库的 v3 版本。我们没有理由相信能够完全消除与 JAX 之间的性能差距，因为作为一个框架，JAX 更适合完全编译的训练循环，但我们怀疑可以大幅缩小差距，同时保持 Pytorch 易于开发和扩展的优势。这些基准测试结果可以根据需要重新计算（使用所有库的最新提交），现在可以在 torchgfn 仓库的 benchmark/文件夹中找到。我们将持续使用这种方法来提高我们库相对于同类库的竞争性能。

表 3: 各阶段迭代时间（毫秒，批大小 128），峰值内存和参数数量。阶段时间取自多个种子的平均值；“—”表示即时编译步骤（不可分解为单独计时）。

Scenario	Library	Sample	Loss	Backward	Optimizer	Total	Mem (MB)	Params
bitseq_small	torchgfn	16.1	2.9	2.7	0.4	22.1	25	76,578
bitseq_small	gfnx	—	—	—	—	1.7	519	74,520
bitseq_medium	torchgfn	9.7	2.9	2.7	0.4	15.8	21	75,554
bitseq_medium	gfnx	—	—	—	—	1.4	519	72,468
box_2d	gflownet	22.4	30.4	7.3	0.5	60.7	22	74,784
box_2d	torchgfn	23.2	3.4	14.9	0.8	42.3	23	82,495
box_2d_uniform_pb	gflownet	22.5	30.5	7.3	0.5	60.8	25	74,784
box_2d_uniform_pb	torchgfn	9.6	2.2	6.2	0.4	18.4	23	74,528
ising_6x6	gflownet	2616.3	929.5	88.2	0.9	3635.0	149	131,367
ising_6x6	torchgfn	72.9	2.9	2.0	0.5	78.3	56	112,530
ising_6x6	gfnx	—	—	—	—	3.9	513	103,020
ising_10x10	gflownet	12413.1	3189.6	244.8	1.5	15849.0	703	246,119
ising_10x10	torchgfn	198.3	5.2	4.0	0.6	208.0	166	194,706
ising_10x10	gfnx	—	—	—	—	9.8	513	168,748
hypergrid_small	gflownet	63.4	62.8	14.3	0.8	141.4	24	70,915
hypergrid_small	torchgfn	25.7	3.0	1.6	0.5	30.8	24	71,430
hypergrid_small	gfnx	—	—	—	—	2.5	182	71,429
hypergrid_medium	gflownet	110.1	79.5	32.7	0.8	223.0	26	83,717
hypergrid_medium	torchgfn	71.6	3.0	2.7	0.5	77.8	43	84,746
hypergrid_medium	gfnx	—	—	—	—	5.9	220	84,745
hypergrid_large	gflownet	197.0	122.2	44.8	0.7	364.7	37	100,101
hypergrid_large	torchgfn	105.0	3.0	4.8	0.5	113.3	55	101,130
hypergrid_large	gfnx	—	—	—	—	10.8	361	101,129

torchgfn: 一个 PyTorch gflownet 库

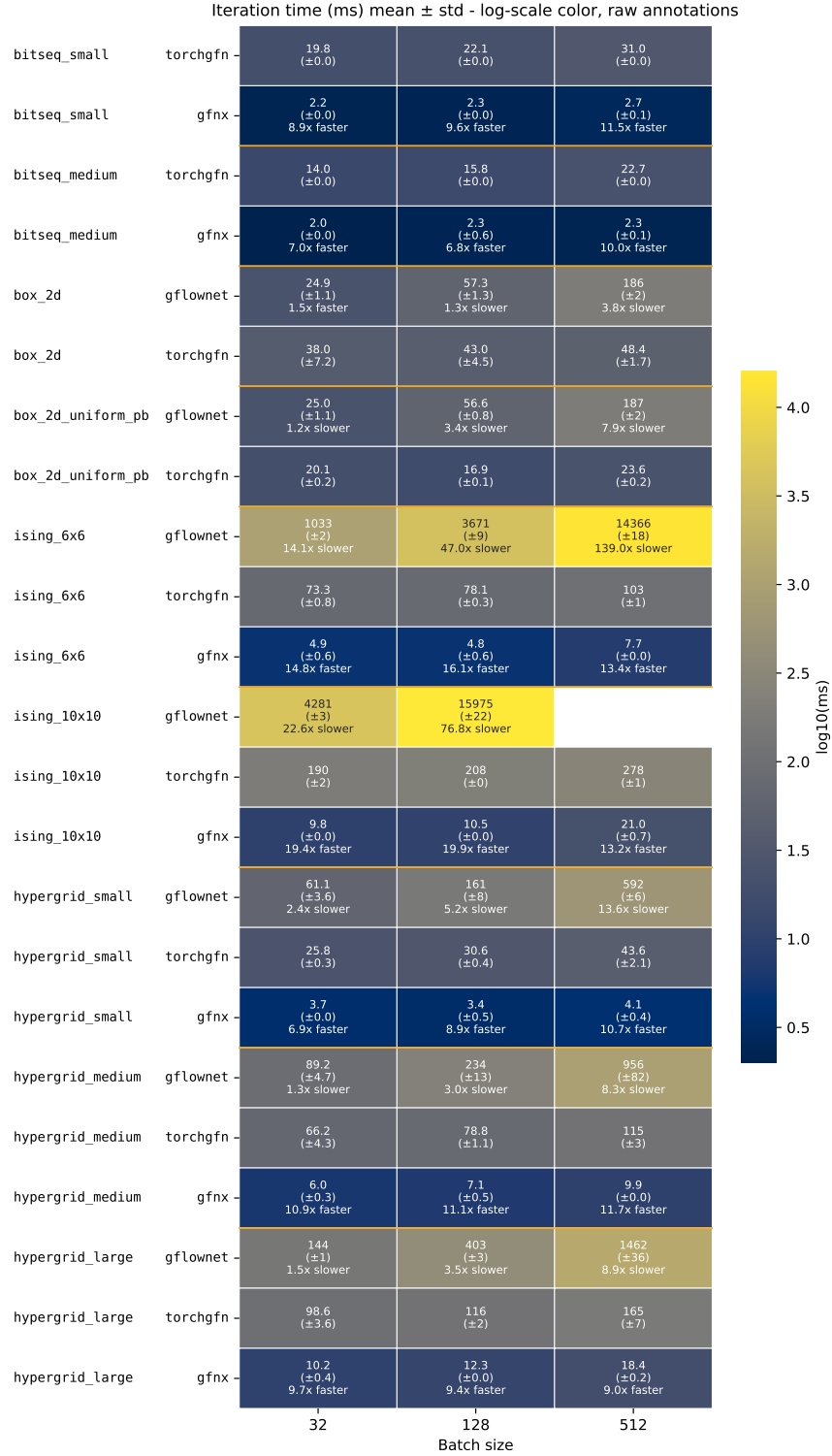


图 3: torchgfn 库每迭代时间 (毫秒) 与 gflownet 和 gfnx 库的比较, 跨越 100 次迭代, 其中包含 50 次迭代的预热期, 基于 3 个种子的方差。由于计算时间难以承受, gflownet 未展示 Ising 10x10 的结果。色谱图以对数尺度呈现, 加速或减速相对于 torchgfn 进行计算。